Core economy systems

Table of contents

Definitions, holders, and managers

Most systems use the same division of responsibility:

Layer	Example	Responsibility
Definition	`Business` ScriptableObject	Authored cost, output, timing, locks, metadata, and progression rules
Holder	`BusinessHolder`	Mutable amount, lifetime amount, bought amount, level, upgrade state, active rounds, stockpile
Manager	`BusinessesManager` component	Holder lookup, buying, merging, production updates, events, reset participation

Definition assets should be treated as immutable rules during normal play. Runtime values belong to holders and are serialized by `SaveFile`.

Use `GetOrAddHolder` when a holder is allowed to be created lazily. Use `TryGetHolder` when you only want existing tracked state. Subscribe to manager events after the managers exist, unsubscribe with the listener's lifecycle, and use `OnHolderAdded` when definitions can gain holders after a screen or scene component is enabled.

Metadata and typed groups

Current assets expose a `Metadata` object with display name, icon, icon string, model, and description. Legacy fields remain hidden for deserialization and are synchronized into `Metadata`.

Metadata is presentation, not identity

`Metadata` supplies user-facing names, descriptions, models, and icons through `IMetadataProvider`, `GetDisplayName`, and `GetIconString`. UI should read this presentation data and fall back to a harmless placeholder when a value is missing. Matching, persistence, and effect targeting use ScriptableObject references and stable asset names; changing a display name must not change the asset name once players have save files.

A business can expose effective metadata for its current upgrade level, and a wrapper inherits presentation from its template unless a supported wrapper-specific value is set. `SyncData` may refresh copied wrapper fields, so keep wrapper presentation in the wrapper's

own metadata fields. For migrated assets, resave and treat `metadata` as authoritative instead of editing legacy name/icon fields in parallel.

Complete typed group family

`ItemGroup<T>` assets create typed relationships without display names or string tags. The concrete types are `BusinessGroup`, `CurrencyGroup`, `BoostGroup`, `UpgradeGroup`, `ManagerGroup`, `LocationGroup`, `AchievementGroup`, and `ShopItemGroup`.

Groups are used for:

- Modifier targeting.
- Aggregate locks.
- Shared business and currency capacity.
- Prestige exclusions. Shop item groups filter shop offers but do not participate in prestige reset exclusions.
- Offer expansion from shop item groups.

An empty modifier target-group list means global. Multiple target groups match any overlapping group. A business assigned to several capped groups is constrained by the smallest remaining capacity. Remove duplicate and null group entries before release.

Current, total, and bought amounts

Do not treat every amount as the same metric:

Amount	Meaning	Typical use
Current amount	What the player owns now	Spending, production, current-amount locks
Total amount	Lifetime amount acquired, independent of later spending or caps	Achievements and lifetime progression
Bought amount	Amount acquired through purchase operations	Geometric cost scaling that ignores free grants
Maximum amount	Effective cap from the asset, business level, upgrades, and group capacity	Capacity locks and purchase limits

For businesses, enable `useGeometricBoughtAmount` when free rewards should not make future purchases more expensive. Old saves without bought/total tracking are seeded conservatively during load; review the migration guide (`EasyIdleGame > md > MigrationGuide.md`).

Purchase previews and source-aware changes

Use `GetPurchasePreview(item, amount)` to render cost, affordability, and the maximum affordable amount without creating a holder or changing state. The returned `PurchasePreview` exposes `canAfford`, `effectiveCost`, `maxAffordableAmount`, and `currentAmount`. `BuyAmount.Type.digit` requests a fixed count; `percent` derives the count from the currently affordable maximum.

A free grant and a purchase are not interchangeable. `TryBuyItems` pays the cost, advances the bought amount, and raises purchase events; `ForceBuyItemsForFree` is for designed grants and does not count as a purchase. Pass the correct `SourceType` with every addition, because statistics, source-aware events, and contextual upgrade filters distinguish purchases, rewards, advertisements, production, and resets.

Currencies do not track a separate bought amount; see Locks and upper locks for the resulting `BoughtAmount` lock behavior.

BigNumber

Economy amounts use `BigNumber` instead of primitive integers or floats. It stores a significand and a large exponent and supports arithmetic, comparisons, geometric sequences, flooring, and abbreviated display.

Use `BigNumber` for authored and runtime quantities. Convert to `double`, `float`, or `int` only at an API boundary where the range is intentionally bounded, such as a UI slider or loop count.

`BigNumber` still has limited precision. It is appropriate for idle-game magnitude, not exact accounting or cryptographic values.

`Mathb` provides the package's geometric-sequence, logarithm, and `BigNumber` range helpers. Its random range helper interpolates between bounds and does not round to an integer.

Safe conversion and geometric pricing

Clamp a `BigNumber` before converting it when the value can exceed the primitive type's range. Formatting is presentation only: never use a formatted amount as an asset key or save identifier.

For a geometrically increasing price, calculate the range from the holder's current or bought amount and the requested purchase amount with `Mathb.GeometricSequence_NthTerm`, `Mathb.GeometricSequence_Sum`, and `Mathb.GeometricSequence_Amount` rather than adding one price per item in a loop. Iterative code becomes impractical when a maximum or percentage purchase represents a very large amount. Format only the final display value.

Inputs and geometric costs

`Input` can consume a business, a currency, or both when both fields are assigned. A list of inputs means every configured input is required.

Buyable assets combine:

- A base `Cost` list.
- `CostMultiplier` for geometric scaling.
- The holder's current geometric index, normally total or bought amount.
- An active purchase-cost modifier.
- Ownership and purchase caps.

Use the holder/manager purchase API so affordability and payment use the same formula. `PurchasePreview` is available where you need a non-mutating UI preview.

The `BuyAmount` model supports two modes:

- `digit` — a fixed count such as 1 or 10.
- `percent` — a percentage of the currently affordable maximum; an entry of `100` acts as a Buy Max option.

`BusinessesManager.buyAmounts` holds the options the player cycles through with `ChangeBuyAmount`; an empty array falls back to buying 1.

Percent options require a meaningful cost. `Business.GetMaxBuyableAmount` throws when the business cost list is empty.

Outputs and drop tables

An `Output` can grant any combination of:

- `businessOutput`
- `currencyOutput`
- `boostOutput`
- `managerOutput`

Every non-null target receives the rolled amount. Keep only the intended targets assigned.

`outputAmount` is the fixed amount or lower bound. When `outputMaxAmount` is greater, the runtime rolls a `BigNumber` between the two values; the result is not automatically rounded to a whole number. Otherwise the output grants exactly `outputAmount` — the default `outputMaxAmount` of `-1` disables randomization.

Each `DropTable` uses one strategy:

Strategy	Behavior
Independent (All)	Checks every output independently with `dropChance`
Weighted Random	Makes `totalDropAmount` picks using relative `weight`; ignores `dropChance`

`DropEvaluationMode` controls how a table behaves when many business copies produce:

- `EvaluateOnce` rolls once and multiplies the resulting batch by the producing amount.
- `EvaluatePerInstance` treats each copy as a roll for batches up to 100 and uses expected-value behavior above 100.

Use Independent for guaranteed outputs plus optional drops. Use Weighted Random for loot-style "pick N" tables.

Previewing rewards without rolling them

Preview code must use the average or expected-output APIs. Do not roll a table to populate a label: rendering must not consume randomness or disagree with the eventual operation. After a claim, purchase, merge, or prestige call succeeds, use its returned `List<Output>` for reward UI and analytics.

Empty tables, and weighted tables with no positive eligible weight, grant nothing; validate any configuration that can consume inputs without producing output. Drop tables are shared by production, level rewards, achievements, daily rewards, prestige, shop items, locations, and merge recipes. `RewardCalculator` performs the common evaluation; the owning manager remains responsible for validating and applying the operation.

Businesses

A `Business` combines buying, unlocking, leveling, upgrading, and production. Important authored areas are:

Area	Key fields
Buying	`cost`, `costMultiplier`, `useGeometricBoughtAmount`, `maxBuyableAmount`
Ownership	`maxAmount_`, `businessGroups`, `location`
Production	`timeToProduce`, `outputTables`, `dropEvaluationMode`, `autoProduce`, `manager`
Production input	`productionCost`, `scaleProductionCostWithAmount`, `allowPartialProductionCost`, `consumeOnProduce`
Rounds	`productionRoundMode`, `autoCollect`, production locks
Progression	player XP per acquisition, business-level thresholds/rewards, `upgradeLevels`
Modifiers	`ignoredMultipliers` and effective typed groups

The effective ownership cap is the most restrictive applicable business cap or group capacity, with active max-business override upgrades able to raise the business-level/base cap according to runtime resolution.

Production round modes

`ProductionRoundMode` controls what happens when ownership changes during production:

Mode	New copies bought during a round
`SingleRoundLiveAmount`	Join the active round; the active output amount refreshes
`SingleRoundSnapshot`	Wait for the next round
`ConcurrentRounds`	Can start in a separate round while existing copies continue

Choose `SingleRoundLiveAmount` for a simple aggregate timer, `SingleRoundSnapshot` when a round must preserve its starting batch, and `ConcurrentRounds` when each newly acquired batch needs independent progress.

The old `increaseProductionAmountAfterRoundEnd` field is migrated to the round-mode enum and remains only for backward compatibility.

ProductionRound lifecycle

Each successful `TryStartProduction` creates a `ProductionRound` that records the copies involved, the progress, and the outputs rolled at the start. Use the round supplied to start/finish events; its outputs were already rolled, so do not roll them again at completion. `ActiveProductions` and `CurrentlyProducing` describe live state, and pausing/unpausing the holder replaces any second project-side timer.

A failed start is normal when copies, locks, capacity, or inputs are unavailable; UI should explain the failing prerequisite instead of expecting an exception. Trigger physical-world effects and custom visuals from completion events rather than duplicating economy timers.

Production costs and consumable producers

Production costs are paid when a round starts.

- With `scaleProductionCostWithAmount` disabled, the base cost is paid once while output can still scale by the producing amount.
- With it enabled, input cost scales by the copies assigned to the round.
- With `allowPartialProductionCost` enabled, the runtime starts the largest affordable producing amount instead of rejecting the entire scaled round.
- With `consumeOnProduce` enabled, producing business copies are removed after their output is applied.

Production-input-cost upgrade blocks can change these costs contextually. Offline profit also accounts for business/currency production inputs and consumable producers.

Collection and stockpiles

With `autoCollect` enabled, completed output is immediately applied.

With `autoCollect` disabled, output enters `BusinessHolder.unclaimedOutputs`. Manual collection applies and clears the stockpile. An unlocked manager can make the offline calculator treat the business as collected automatically, depending on the manager automation state.

Use a stockpile when the player should return to claim output. Use immediate collection when the resource should flow directly into chains or purchases.

Business levels and upgrade levels

Business level progression and business upgrade levels are separate mechanisms:

- **Business level** uses owned-copy thresholds, XP-like progression, descriptions, and level rewards.
- **Business upgrade level** uses `IUpgradableHolder` progression and can override metadata, maximum ownership, and maximum buyable amount.

`LevelDataHolder` combines `LevelDescription` entries, exact `LevelRewardHolder` entries, and repeating level rewards. `UpgradeLevel` is the common upgrade-level model; `BusinessUpgradeLevel` adds business-specific metadata and cap overrides.

`BusinessUpgradeLevel` metadata falls back to the base business metadata for empty values. A `-1` maximum means unlimited.

Level rewards use `Reward` assets and drop tables. The old standalone level-reward format was replaced in version 1.6 and is not asset-compatible with the earlier layout.

Business groups and capacities

Create a `BusinessGroup` when multiple businesses need a shared target or shared cap.

- Assign the group to each business.
- Set a non-negative group capacity to cap the sum of current amounts across the group.
- Target the group from upgrades, boosts, filters, locks, or prestige exclusions.

Wrappers use effective groups inherited from their underlying business when appropriate. Capacity checks and offline profit use these effective groups.

Legacy string groups remain a fallback only. Do not combine new typed targeting with assumptions that the string field will be ignored in every compatibility path.

Business wrappers

`BusinessWrapper` copies an `underlyingBusiness` and then applies independent scales for:

- Buying cost.
- Production cost.
- Upgrade cost.
- Production time.
- Outputs and level rewards.

What SyncData copies

`SyncData()` deep-copies serialized lists, applies the configured scales, remaps self-references from the template to the wrapper, resolves effective metadata and groups, and keeps runtime holder state isolated. It runs on enable and validation and is also available from **Force Sync Data**. Because synchronization can run during validation, wrapper-specific edits to copied fields can be replaced the next time it runs.

When to use a standalone business

Use a wrapper when variants share a canonical definition and differ only through the supported scales or presentation overrides. Use a standalone `Business` when costs, outputs, production rules, levels, or references must diverge structurally. Never set a wrapper as its own template. After changing the template, resync and inspect each wrapper's rewritten self-references before saving the assets.

Merge recipes

`BusinessMergeRecipe` consumes a list of normal `Input` entries and applies output `DropTable` entries. Inputs may be businesses or currencies; outputs may be businesses, currencies, boosts, or managers.

Use:

```
if (BusinessesManager.Instance.TryMergeBusinesses(recipe, out List<Output> outputs))
{
    // Update the merge UI with the actual rolled outputs.
}
```

The recipe also stores base merge duration and cooldown values for games that implement timed merging. Contextual upgrade blocks can target the recipe and modify merge input cost, effective merge speed, or merge cooldown. `UpgradeType.speed` shortens the effective merge duration, while `UpgradeType.cooldown` multiplies the merge cooldown (values below 1 shorten it).

Legacy arrays of business-only merge holders are migrated into normal inputs and `DropStrategy.All` tables during deserialization.

Lookup, execution, and visual-game boundary

Keep recipe assets below a `Resources` folder for discovery and call `BusinessesManager.RefreshMergeRecipeCache()` after changing the runtime catalog. `BusinessesManager.TryGetMergeRecipe` finds a recipe for two businesses or for arbitrary inputs; duplicate inputs must account for the total quantity required from the same target.

Empty inputs create a free recipe and empty outputs create a destructive recipe; validate both intentionally. The package validates affordability, applies contextual input-cost and speed effects, and returns the rolled outputs — it does not move board pieces, animate merges, or run merge timers. `mergeDurationSeconds` and `mergeCooldownSeconds` are authored data plus effective-value helpers; project code implements the timers and visuals. Apply the economy operation through the manager, then update the visual board from the confirmed result.

Currencies and currency groups

A `Currency` defines metadata, typed groups, and a maximum amount. `CurrencyManager` tracks current and lifetime totals and raises amount-added events with `SourceType`.

Currency groups can impose a shared capacity. Additions are clamped by the currency cap and all relevant group capacities. An owned `maxCurrencyAmount` override upgrade can raise the applicable asset limit; the highest modified override wins.

Use `GetCurrencyAmount` for the current balance and `GetTotalCurrencyAmount` for lifetime acquisition.

CurrencyHolder, source events, and effective capacity

Add and remove currency through `CurrencyManager` so caps, `SourceType`, player statistics, save-worthy notifications, and amount events stay coherent. Subscribe to `OnCurrencyAmountChanged` for normal UI refresh instead of polling unchanged labels.

Query `GetMaxCurrencyConstrain(currency)` for the effective limit before converting an amount for a Unity-facing API. Rewards and offline profit can be truncated by the individual or group cap, so final grant UI should display the applied result, not only the calculated amount.

Locks and upper locks

A `Lock` may combine several requirements. Every assigned requirement must pass.

Supported targets include:

- A business, currency, or business upgrade level.
- Player level, custom stat, or prestige count.
- Active location.
- Business, currency, or upgrade group totals.

For business and currency requirements, `LockAmountType` selects:

- `CurrentAmount`
- `TotalAmount`
- `BoughtAmount`
- `MaxAmount`

For currencies, `BoughtAmount` currently aliases the current balance because currencies do not track a separate bought count.

Normal locks are lower bounds. Upper locks invert amount comparisons and are useful for hiding early-game content after a threshold. Location upper-lock behavior is based on the location not being active.

When `unlockOnlyOnce` is enabled, a holder remembers that its lower locks passed. Upper locks are still evaluated after that point. Prestige has separate flags for resetting the cached lower-lock state per system.

Every configured manager-backed requirement depends on its matching manager. `IsUnlocked` and `IsUpperUnlocked` validate all configured dependencies before evaluating values and throw an `InvalidOperationException` naming the requirement and missing manager when the scene is misconfigured. Add every manager referenced by a lock.

Production locks and auto-unlock

Manager auto-unlock periodically checks eligible items and raises the normal unlock events. A business can also have production locks and upper production locks that are independent of its purchase/unlock state; when production fails, inspect both the availability conditions and the production conditions before checking inputs.

If a requirement is not representable with a built-in lock, expose it through a `CustomStat` or keep the project-specific availability check outside the package lock asset.

Rewards and source types

`Reward` is a reusable asset containing one or more drop tables. Rewards are used by level progression, achievements, daily rewards, prestige, shops, and other flows.

`SourceType` describes why an amount or effect was created:

`Any`, `Generic`, `RecurringReward`, `Purchase`, `Advertisement`, `Milestone`, `Reset`, `Conversion`, `Event`, `RandomDrop`, and `Production`.

Source types are propagated through many amount-added events and contextual upgrade filters. Use them when the same currency or output needs different modifiers for production, a purchase, a daily calendar, or a reset reward.